2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3315-4497-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/COMPSAC69091.2026.00027

Toward Standardized Evaluation of Metamorphic Relations: A Structured Rubric and Human–LLM Comparison

Yifan Zhang*§, Dave Towey†¶, Matthew Pike†, Quang-Hung Luu‡, Huai Liu‡, Tsong Yueh Chen‡
*Ningbo Ningshu Intelligent Computing Industry Development Co., Ltd., Ningbo, Zhejiang 315048, China
†School of Computer Science, University of Nottingham Ningbo China, Ningbo, Zhejiang 315100, China
‡Department of Computing Technologies, Swinburne University of Technology, Hawthorn, VIC 3122, Australia

***Abstract*—Software testing often depends on a test oracle to determine whether or not an output is correct. However, many real-world systems lack reliable or affordable oracles. Metamorphic testing (MT) alleviates this problem by using *metamorphic relations* (MRs), which examine the correctness of relations between outputs produced from related inputs across multiple executions. While the effectiveness of MT depends heavily on the quality of MRs, existing MR evaluation is typically guided by informal or study-specific criteria, limiting standardization and repeatability, especially for complex and AI-integrated systems.**

This paper proposes a structured MR-evaluation rubric that operationalizes key criteria to support consistent and repeatable assessment. We conducted an empirical study on nine systems under test (SUTs) spanning three levels of complexity, using fresh sets of LLM-generated MRs for each SUT. We then compared evaluations from human experts against two widely used large language models (LLMs), namely ChatGPT and Gemini, by configuring these models as MR evaluators using the same rubric. The results show that human and LLM evaluators are more consistent when assessing simpler SUTs, but their judgments increasingly diverge as system complexity grows and evaluation criteria become more varied. The study further reveals clear differences in LLM-evaluation behaviors, indicating that evaluator choice can affect MR assessment outcomes in complex settings. Overall, the proposed rubric provides a standard framework for MR evaluation, and supports a systematic comparison between human and LLM evaluators, offering empirical evidence on when LLM-based MR evaluation is more reliable, and where additional care is needed as SUT complexity increases.

***Index Terms*—Metamorphic testing (MT); metamorphic relation (MR); oracle problem; metamorphic relation evaluation; ChatGPT; Gemini; large language model (LLM)**

I. Introduction

Ensuring the correctness and reliability of modern software systems remains a central challenge in software testing, particularly when traditional test oracles are unavailable, expensive to construct, or inapplicable in practice [1]. A test oracle provides a reference for expected results [2]. The oracle problem refers to the situation when the correctness cannot be practically verified by examining system outputs [1]. It is especially common in complex, data-driven, or highly configurable systems. Metamorphic Testing (MT) offers a practical solution to alleviate the oracle problem through examining relations among multiple executions. These relations, known as Metamorphic Relations (MRs), are derived from the necessary properties of the systems under test (SUTs). They specify the behaviors of its outputs when the inputs are transformed [3]. MT has been shown to be effective across a wide range of complex software [1], including autonomous driving systems (ADSs) [4], simulators [5], compilers [6], numerical computations [7], multiple linear regression programs [8], and machine translation models [9].

However, the effectiveness of MT largely relies on the quality of the MRs [3]. Poorly specified or incorrect MRs can lead to tests that are impossible to automate, difficult to interpret, or fundamentally invalid [1]. For this reason, MR evaluation is a prerequisite for effective MT, since an MR that is not correct cannot meaningfully support further judgments about usefulness, transferability, or practicality.

While MT has a substantial amount of work on identifying, constructing, and automating MRs [1], few studies have addressed the quality of MRs [10]. Evaluation of MRs traditionally relies on criteria developed from empirical research and expert insights [1]. However, such criteria often lack the depth and breadth needed to effectively assess MRs in the context of increasingly complex software environments. This lack of comprehensive evaluation methodologies highlights the need for further research and development in this area. Li *et al.* [10] proposed *Adequacy* and *Diversity* as theoretical foundations for MRs. Adequacy concerns whether or not a set of MRs provides sufficient and meaningful test power [10]. For instance, whether or not the relations fully reveal important behaviors, constraints, and failure modes of the SUT. Diversity concerns whether or not the MRs are varied, rather than redundant: Whether or not they cover different input transformations, behavioral properties, operating conditions, and system functionalities [10]. They further noted that these concepts require more systematic formulations from multiple perspectives, motivating structured and operational evaluation approaches [10].

This paper targets this gap by focusing on *MR evaluation* with Large Language Models (LLMs). LLMs are increasingly used across software engineering tasks [11], such as code generation, test-case design, and debugging [12]. In MT re-

§Part of this work was completed while the first author was at the University of Nottingham Ningbo China.
¶Corresponding author.

2836-3795/26/$31.00 ©2026 IEEE
DOI 10.1109/COMPSAC69091.2026.00027

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

search, LLMs have been explored as assistants for generating MRs [13], [14], [15]. However, two key questions remain underexplored: Can LLMs be used as MR evaluators, and how do their judgments compare with those of experienced human testers under a unified rubric, especially as system complexity increases?

Building on an earlier MR-assessment study [13], we present a structured evaluation rubric designed to improve standardization and accuracy of the process by specifying abstract criteria into explicit, checkable assessment items. We compared MR evaluations produced by human experts and multiple LLM evaluators across three SUT categories of increasing complexity: basic computational functions; complex systems without AI components; and complex systems with AI components. Each category contains three SUTs. For each SUT, we asked ChatGPT to generate a fresh set of MRs, rather than reusing existing MRs, leveraging its demonstrated MR-generation capability [14]. This design also reduces threats to validity that could arise if the MRs were written by human participants or drawn from sources that may have been seen during LLM training.

To support a representative comparison across widely used LLM platforms, we selected ChatGPT (GPT-5.2) [16] and Gemini-3 [17] as the base models for LLM evaluators. This setup enables us to examine whether MR-evaluation behavior varies across leading model families, rather than attributing observed patterns to a single LLM. Specifically, we investigate how MR-evaluation results produced by LLM evaluators compare with those from experienced human testers under a unified rubric when the SUTs range from basic deterministic programs to complex systems, including AI-integrated ones.

In summary, this work makes the following contributions: (1) we propose a structured and operational rubric for MR evaluation to support repeatable assessment; (2) we report on an empirical comparison between human experts and LLM evaluators across SUT categories spanning different levels of complexity; and (3) we provide an analysis of evaluator behaviors, and their implications for reliable MR assessment.

The rest of the paper is organized as follows: Section II outlines the evaluation rubric for assessing MRs. Section III introduces the experimental settings about asking ChatGPT to generate MRs for SUTs and configuring LLM evaluators to evaluate the MRs along with the human experts under the same rubric. Section IV lists the experiment results, and Section V summarizes the findings and discusses the limitations of the study. Finally, Section VI concludes the findings, along with the future work we recommend to extend the study.

II. Evaluation rubric for assessing MRs

An earlier study [13] introduced a set of preliminary scoring criteria to assess the quality of LLM-generated MRs. However, it also noted that the criteria could be subjective and not fully systematic in coverage [13]. Motivated by these limitations, and the need for a more comprehensive and repeatable evaluation procedure, we have refined the framework into seven explicit criteria (Table I): Completeness, Correctness, Generalizability, Novelty, Clarity, Computational Feasibility, and Applicability.

In particular, each criterion is decomposed into explicit sub-criteria such that evaluators can make decisions based on concrete checks rather than broad impressions. For instance, the *completeness* and *correctness* criteria are defined to strictly verify the structure of the input and output relations, ensuring they provide a clear blueprint for test-case generation and validation. Based on these sub-criteria, the rubric assigns each criterion a 0–3 scoring scale (a single score is assigned per criterion; score levels are cumulative rather than additive), except for *Completeness*, which is scored on a 0–1 scale. The evaluation rubric is detailed in the following subsections.

A. Completeness

The "completeness" criterion determines whether the MR includes all essential components.

- Presence of key components: The MR must contain the following critical elements (1 mark):
  - Source Test Case: Well-defined source test cases that serve as the basis for the MR.
  - Input Relation: A clear and explicit input relation that specifies how follow-up test cases are generated or modified from the source test case.
  - Output Relation: A detailed output relation that describes the expected relationships between the outputs of follow-up test cases and the source test cases.
- Zero marks for missing components:
  - If the MR lacks any of these key components (source test cases, input relation, or output relation), it will be assigned zero marks for completeness.
  - This criterion is binary: the MR either contains all required components (1 mark) or is missing one or more components (0 marks).
  - *This criterion is considered to be the baseline of MRs. If the MR scores zero marks for completeness, it is considered incomplete or insufficient for evaluation. As a result, it will also receive zero marks under all the other evaluation criteria.*

B. Correctness

The "correctness" criterion assesses whether or not the MR accurately represents the intended behavior of the SUT.

- If the input relation and output relation violate the SUT's specifications, this criterion will score zero marks.
- The input relation correctness (1 mark):
  - The MR's input relation must focus on the current SUT. This means that it should not alter the fundamental nature or category of the SUT through its input modifications.
  - The input relation should explicitly describe how subsequent test cases are derived from the original source test case. This includes clear methodologies or rules for test-case generation, ensuring predictability and repeatability.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Overview of the evaluation rubric

Criteria	Criteria Details	Maximum Points
Completeness	Includes Source Test Case, Input Relation, Output Relation	1
Correctness	Assesses Input and Output Relation Adherence, Overall Behavioral Correctness	3
Generalizability	Evaluates Suitability for Different SUTs: Current, Specific Group, All Types	3
Novelty	Assesses Originality in Input/Output Relations, Overall Metamorphic Relation	3
Clarity	Evaluates Clarity Levels: For Experts, Basic Understanding, General Audience	3
Computational Feasibility	Examines Generation Ease, Automation of Test Cases, MR Validation	3
Applicability	Focuses on Relation to SUT's Key Features: None, Partial, Strong	3

- The output relation correctness (2 marks):
  - The MR's output relation should be free from ambiguous or uncertain language such as "may" , "could", or similar terms. It should convey clear and definite relationships.
  - The output relation must clearly articulate the expected relationships between the outputs of follow-up test cases and the source test cases. This involves specifying the expected changes or consistencies in outputs, based on the input variations.
- Overall correctness (3 marks):
  - The MR should focus on the relationships among multiple test cases, rather than mirroring the SUT specifications or assessing the correctness of individual test cases.
  - This criterion emphasizes the MR's role in revealing expected behaviors or properties through a clear, checkable relationship among multiple test cases, rather than simply reiterating what is already specified about the SUT.
- This criterion is also considered to be a baseline for MRs. If the MR scores zero marks for correctness, it is considered incorrect for evaluation. As a result, it will also receive zero marks for all the other evaluation criteria.

C. Generalizability

This criterion evaluates the MR's suitability across different SUTs.

- Generalizability to certain SUTs (1 mark):
  - The MR is specifically tailored to the current SUT and cannot be directly applied to other systems without significant modifications.
  - The characteristics or requirements of the MR are unique to the current SUT, such as relying on specific hardware, software, or configurations not commonly found in other systems.
- Generalizability to certain groups of SUT (2 marks):
  - The MR is generalizable to a specific group of SUTs within a certain type or category. This means that the MR can be used with systems that share certain characteristics or functionalities, but is not universally applicable across all systems of that type.
  - There exist other SUTs within the same category to which the MR cannot be applied, indicating a level of specificity.
- Generalizability to all types of SUT (3 marks):
  - The MR can be applied universally to all known SUTs within the same category, demonstrating a high level of adaptability and generalizability.
  - This level of generalizability implies that the MR is not constrained by specific features, configurations, or special requirements that are unique to certain SUTs.

D. Novelty

The "novelty" criterion measures the originality and uniqueness of the MR by comparing it with previously established works and knowledge in the field. In this study, such comparison refers to checking whether the MR closely matches known metamorphic relation input patterns (MRIPs)[1], metamorphic relation output patterns (MROPs)[2], metamorphic relation patterns (MRPs)[3], or representative MRs reported in prior MT studies. For instance, MRs based on common mathematical properties, such as sine periodicity or adding zero to a summation input, would normally be judged as having low novelty. In contrast, an MR targeting system-specific behavior, such as the stability of an autonomous-driving perception system under changed lighting conditions, may be judged as more novel.

- Novelty in the input relation (1 mark):
  - The input relation of the MR does not align with any established MRIPs. This means it introduces a new way of altering or creating inputs that are not already documented.
  - The input relation is distinct and not easily comparable to examples found in existing literature or knowledge in the field.
- Novelty in the output relation (2 marks):
  - The output relation of the MR deviates from known MROPs. It suggests an innovative approach to interpreting or utilising the outputs of an SUT.
- Novelty in the overall MR (3 marks):

[1]MRIP is an abstract representation of the connections between the initial and subsequent inputs within a potentially infinite collection of MRs [18].

[2]MROPs define a set of abstract relationships that focus on the outputs of MRs [19].

[3]The MRP describes a set of MRs that share the same level of abstraction [18].

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

  - The MR as a whole (including both its input and output relations) does not fit within any recognised MRPs. This indicates a completely new approach or method that has not been explored before.
  - The MR exhibits distinct originality, which indicates no substantial parallels in terms of the selection of source test cases, approach to construct follow-up test cases, and comparison of outputs when compared to existing works and knowledge.

E. Clarity

This criterion assesses how easily the MR can be understood by different audiences.

- Clarity for domain experts (1 mark):
  - The MR assumes a deep understanding of advanced concepts specific to the field.
  - It requires familiarity with specialized jargon and technical terminology commonly used among experts
  - The MR relies on knowledge of specific methodologies and practices typically known only to those with extensive experience or specialised education in the domain.
- Clarity for individuals with basic understanding (2 marks):
  - The MR is understandable by individuals with basic knowledge of the field, such as students who have completed relevant introductory courses.
  - It is also suited for professionals in related fields who possess general knowledge of the topic, but not in-depth specialisation.
  - While extensive experience is not required, the MR assumes the reader has some fundamental understanding of the subject.
- Clarity for general audiences (3 marks):
  - The MR is written to be accessible to a general audience, requiring no specific background knowledge in the field.
  - Technical terms, if used, are clearly explained to ensure understanding by those unfamiliar with the field.
  - The MR is structured in a clear, logical manner, making it easy to follow for someone with no prior exposure to the subject matter.

F. Computational feasibility

The "computational feasibility" criterion assesses the ease and practicality of applying the MR in computational terms.

- Ease of generating source test cases (1 mark): The MR facilitates the generation of source test cases with minimal complexity. This implies that:
  - The process for generating test cases is straightforward and does not require advanced coding skills or specialised software.
  - It can be accomplished using standard tools and techniques commonly used in the field.
  - The generation process is efficient, with a low requirement for computational resources and time.
- Feasibility of automating test case generation (2 marks): The generation of test cases can be fully automated through code. This level of automation means:
  - The process can be executed consistently by a script or program without human intervention.
  - The automation is replicable, capable of producing consistent results across multiple executions.
  - The system can adapt to various inputs or changes in requirements, with minimal need for reprogramming.
- Feasibility of automating MR validation (3 marks): The validation of the MR can be completely automated through code. This involves:
  - Developing scripts or programs that can automatically perform the entire process of MT (generating test cases, and comparing outputs against MRs).
  - The automation is robust, reliably handling different scenarios and edge cases.
  - The code is efficient in terms of computational resources and time, and operates without the need for manual oversight or intervention for reliable outcomes.

G. Applicability

This criterion evaluates how closely the MR focuses on the key features of the SUT.

- No relevance to key features (1 mark): The MR addresses aspects that are generic and not specific to the SUT. Characteristics of this level include:
  - The MR applies to a wide range of SUTs across different categories, indicating a lack of specificity.
  - Neither the input nor the output relations of the MR are uniquely tied to the characteristics or functions of the SUT.
  - Example: Running the program multiple times results in consistent outputs, regardless of the SUT's unique features.
- Partial relevance to key features (2 marks): The MR demonstrates a limited connection to the SUT's unique aspects. This is characterised by:
  - The MR is somewhat specific to the SUT while not fully reflecting its unique characteristics.
  - Either the input or the output relation of the MR is specific to the SUT, but not both.
  - Example: Increasing the number of actors in a driving scenario does not affect the response time of the self-driving system, showing a specific, but limited, relation to the SUT's behavior.
- Strong relevance to key features (3 marks): The MR closely relates to and highlights the unique features of the SUT. This includes:
  - The MR focuses on aspects that are uniquely identifiable with the SUT, reflecting a deep understanding of its characteristics.
  - Both the input and output relations of the MR are specifically tailored to the SUT, reflecting its distinctive features and behaviors.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

– Example: Add a controlled amount of sensor noise to the driving scenario (e.g., perturb the camera/LiDAR measurements) and check that the SUT still detects obstacles in the area of interest, such that the obstacle count does not drop substantially compared with the baseline. This directly targets the SUT's noise robustness and obstacle-detection behavior.

III. Methodology

A. MR generation

Since LLMs have been proven effective in MR generation [5], [13], [14], we asked ChatGPT to generate MRs for testing nine target systems across three SUT categories of increasing complexity, designed to cover different levels of complexity and AI usage, as shown in Table II. The first group, basic computational functions (Systems 1–3), consists of SUTs governed by fundamental, deterministic algorithms. The second group, complex systems without AI components (Systems 4–6), includes software that performs intricate data processing and analysis, but does not use AI models. The final group, complex systems with AI components (Systems 7–9), was selected to evaluate the MRs for inherently non-deterministic, AI-powered systems.

To answer whether or not LLMs can replace human experts in MR evaluation, we developed two distinct LLM MR evaluators based on ChatGPT (GPT-5.2 [16]) and Gemini (Gemini-3 [17]). These evaluators were configured using the evaluation rubric, which can improve both accuracy and efficiency by constraining the model's response scope to rubric-relevant judgments [20]. We used the free versions of these LLMs to reflect a common access setting for general users, while acknowledging that performance may differ across model versions, access tiers, and deployment configurations.

B. Prompt methods

In this study, we used zero-shot prompting [21] to ask ChatGPT generating MRs. Zero-shot prompting involves interacting with the model without providing explicit examples or prior context [21]. The operation was performed in two steps: First clarifying with the LLM about the target system, and then asking it to generate a set of unique MRs for each target system in separate chat sessions. These MRs were generated by specifying the main inputs and outputs of the programs (as presented in Table II) to ChatGPT, enabling it to understand the functional scope and the data types it was dealing with. An example of such a prompt is as follows:

> "Generate three MRs for an ADS perception program. The main inputs are images and point clouds, and the main outputs are object detection results."

The MRs generated by ChatGPT (in this study) can be found in Table IV in the Appendix.

C. Configuring LLM MR evaluators

The configuration of an LLM evaluator is a straightforward and structured process. The detailed configurations are outlined as follows:

1) Define the evaluator's role:
   The first step is defining the LLM's role as an MR evaluator and regulating the scoring procedure. The evaluation follows a level-based scoring method for each criterion, where meeting the corresponding level of a criterion results get its corresponding points, and failure to meet it results in zero points. An example of how the LLM evaluator was configured in this study is provided below:
   > *In your role as an MR evaluator, you focus on evaluating metamorphic relations (MRs) in software testing, using a set of criteria. Each MR is assessed based on input relation and output relation aspects. The scoring for each criterion follows the defined rubric levels: if the corresponding level is met, the MR earns the points for that level; if not, it scores zero.*
2) Incorporate the evaluation criteria:
   The evaluation criteria (Section II) are then inputted directly beneath the role definition. This ensures that the criteria are clear, accessible, and directly linked to the evaluator's role.
3) Set answer formats and components:
   To improve the efficiency of analyzing results, and to standardize the process, the evaluator is expected to begin with a summary table displaying scores for each criterion, followed by a comprehensive justification for these scores. An example used in this study is provided below:
   > *Your evaluations should start with a summary table of scores for each criterion, followed by detailed justifications. Maintain clarity and precision in your explanations, aligned with the latest software testing standards.*

Following this configuration approach ensures that the LLM evaluator can effectively prepare to assess MRs. Fig. 1 shows an example of the evaluator's outcomes and explanations.

D. Result analysis methods

After gathering data from both the human experts and the LLM evaluators, we obtained the findings by calculating the average scores for SUTs within each category. This process enables a comparative analysis between the evaluations conducted by humans and by LLMs, highlighting similarities and differences in the assessment of MRs. Such a comparison provides a comprehensive and objective perspective on the quality of the MRs. Furthermore, by analyzing the scoring tendencies of both human evaluators and LLMs, we can summarize the unique patterns and characteristics in each evaluator's approach to assessment.

IV. Evaluation Results

Table III presents the average MR evaluation scores from both human experts and LLM-based evaluators, and Fig. 2a to Fig. 2c visualize their scoring results via radar charts. Overall, evaluation results tended to diverge as SUT complexity

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Details of the SUTs in this study

ID	System	Description	Main Inputs	Main Outputs	Category
1	SIN	Computing *sine*	One number	One number	Basic computational functions
2	SUM	Computing *sum*	A list of numbers	One number	Basic computational functions
3	SHORTEST-PATH	Finding the shortest path	A graph with vertices, edges	A path	Basic computational functions
4	REGRESSION	Multiple linear regression	Multiple data row	Coefficients, predicted data	Complex systems without AI components
5	FFT	Fast Fourier Transform-based analysis	Time-series of data	Frequencies, amplitudes	Complex systems without AI components
6	WFS	Weather forecasting system	Multiple sources	Multiple outputs	Complex systems without AI components
7	AV-PERCEPTION	Autonomous vehicle perception	Images, point clouds	Object detection	Complex systems with AI components
8	TRAFFICSYS	AI-based traffic light control	Sensor data	Traffic decision	Complex systems with AI components
9	AUTOPARKING	Autonomous vehicle parking	Vehicle location, obstacles	Parking trajectory, decisions	Complex systems with AI components

TABLE III: MR evaluation results

Category	Evaluator Type	Completeness	Correctness	Generalizability	Novelty	Clarity	Computational Feasibility	Applicability	Total (Percentage Score)
Basic computational functions	human	1.0	2.5	2.8	1.2	2.6	2.7	2.8	15.7 (82.6%)
	ChatGPT	1.0	2.9	2.8	0.4	2.7	2.9	2.9	15.6 (82.1%)
	Gemini	1.0	2.4	2.7	0.1	2.7	2.7	2.7	14.2 (74.7%)
Complex systems without AI components	human	1.0	2.0	2.8	2.0	2.2	1.9	2.8	14.7 (77.4%)
	ChatGPT	1.0	2.0	2.3	1.0	2.0	2.4	2.6	13.3 (70.0%)
	Gemini	1.0	1.9	2.2	1.2	2.2	2.2	2.1	12.9 (67.9%)
Complex systems with AI components	human	1.0	1.9	2.6	1.9	2.1	1.9	2.7	14.1 (74.2%)
	ChatGPT	1.0	1.9	1.8	0.9	2.2	2.3	2.4	12.6 (66.3%)
	Gemini	1.0	0.3	0.4	0.6	0.6	0.7	0.6	4.1 (21.6%)

increased, but the extent and direction of divergence also depended on the evaluator.

In the "basic computational functions" category, the percentage scores[4] were relatively tightly aligned across evaluators: human (82.6%), ChatGPT (82.1%), and Gemini (74.7%). Notably, ChatGPT was more closely aligned with the human score than Gemini, which is also reflected in the radar diagram (Fig. 2a) as greater overlap between the human and ChatGPT scores across criteria. As the figure shows, the evaluators assigned similar marks on most criteria. However, on the *novelty* criterion, divergence remains evident: ChatGPT (0.4) and Gemini (0.1) rated novelty lower than humans (1.2), indicating that novelty judgments are sensitive to evaluator style, even when overall assessments are broadly consistent.

A similar divergence was also found in the "complex systems without AI components" category (Fig. 2b). When normalized to percentage scores, ChatGPT achieved 70.0% and Gemini 67.9%, whereas the human score was 77.4% (14.7), with ChatGPT more closely aligned than Gemini. The reductions concentrated in *generalizability*, *novelty*, and *applicability*. Meanwhile, both ChatGPT and Gemini assigned higher marks than humans for *computational feasibility*, indicating that the LLMs tended to rate the practical difficulty of executing and automating the MRs as lower compared with human experts.

The most obvious divergence existed in the "complex systems with AI components" category (Fig. 2c), where the LLMs showed markedly different behaviors. After normalizing the total marks to percentage scores, it can be found that

[4]We use percentage scores to normalize results onto a common 0–100 scale, making comparisons across evaluators more interpretable and robust to the absolute magnitude of the raw totals. The percentage score was computed as $\frac{\text{total score}}{\text{maximum possible total score}} \times 100\%$. In our rubric, the "maximum possible total score" is 19.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

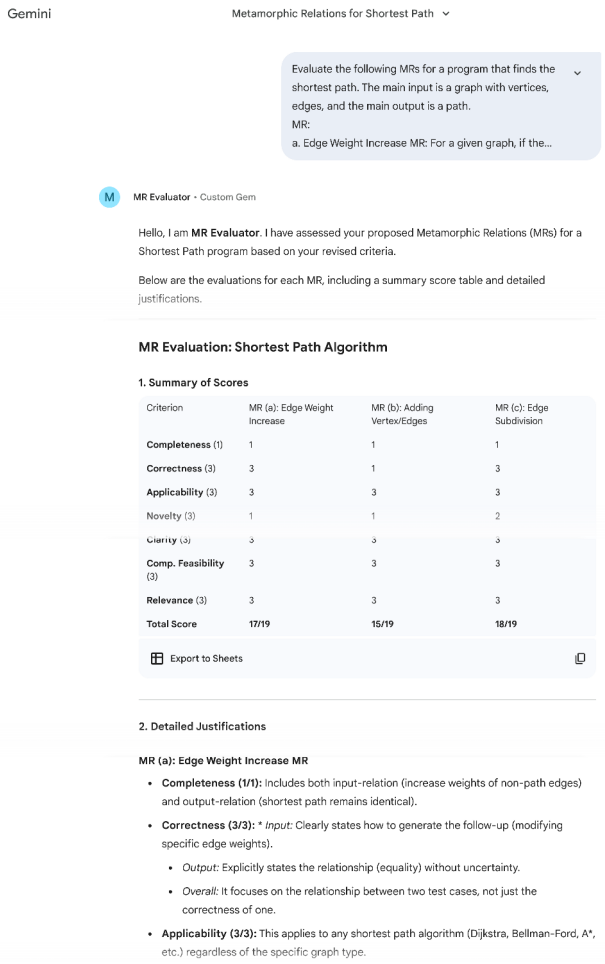
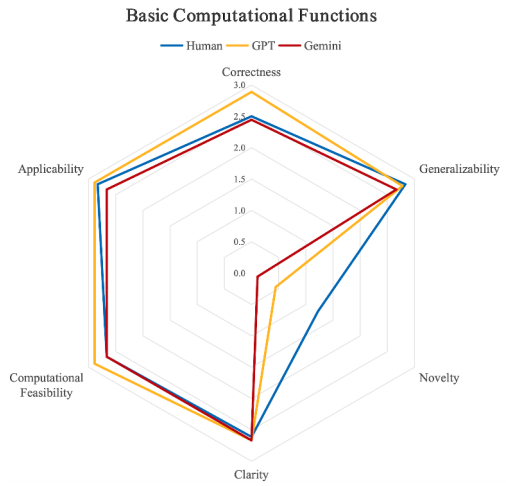
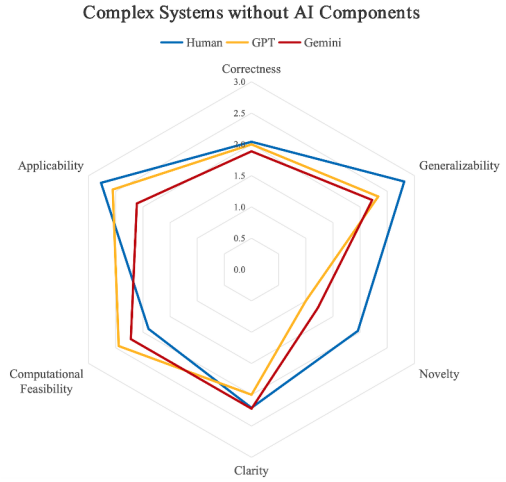


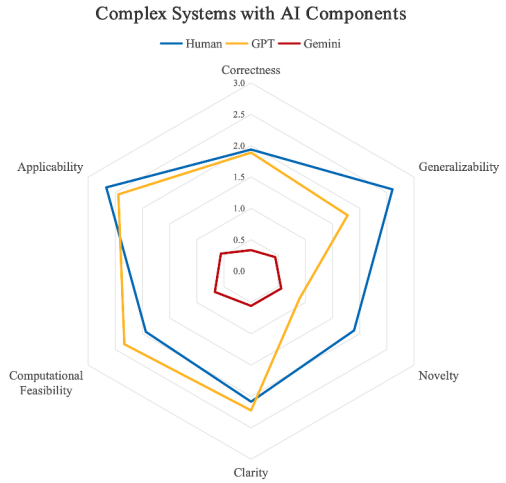
Fig. 1: An example snippet of asking the LLM evaluator to evaluate MRs



(a) Evaluation results of basic computational functions



(b) Evaluation results of complex systems without AI components



(c) Evaluation results of complex systems with AI components

Fig. 2: Radar diagrams of evaluation results

ChatGPT (66.3%) stayed relatively close to the human total (74.2%), tending to score higher on *computational feasibility*, while being more conservative on *generalizability*, consistent with the observations. Gemini, however, produced a very low total (21.6%), primarily due to a near-zero *correctness* score (0.3), which, in accordance with the evaluation rules (Section II-B), resulted zero scores for all the other criteria (e.g., *generalizability* and *applicability*), thereby driving the average marks toward zero. Taken together, these results suggest two distinct evaluation styles: Gemini follows the guidelines most strictly, whereas ChatGPT's scoring is closer to human judgments, but still shows systematic differences on criteria such as *generalizability* and *computational feasibility*.

V. Discussion

A. Comparative analysis of evaluation styles between human and LLMs

Through examining the evaluation data, the main differences are not only in the overall scores, but also in how evaluators

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

interpret the criteria. For basic computational functions, where programs are composed of clear, deterministic algorithms, all evaluators produced relatively aligned total scores, indicating a shared baseline for assessing MR quality. Even so, the *novelty* criterion already differentiates evaluators: ChatGPT and Gemini assigned lower novelty scores than the human evaluators. This indicates that novelty judgments are particularly sensitive to evaluator style, even when overall assessments remain broadly consistent.

As the SUT becomes more complex, the divergence becomes more obvious and is driven mainly by criteria that require broader interpretation, such as *generalizability*, *novelty*, and *applicability*. Human experts tend to assign higher marks on these dimensions based on testing experience and anticipated impact, whereas LLMs can be more conservative when the MR does not explicitly justify transferability or novelty.

This observation aligns with another cross-category pattern: LLMs often assign higher marks for *computational feasibility* in complex settings. The difference likely reflects what "feasible" means in practice. Human experts typically consider end-to-end engineering constraints (instrumentation, data access, controllability, reproducibility), whereas LLMs may treat feasibility as conceptual automability. The gap becomes more obvious for AI-integrated systems, where outputs are less predictable and algorithm logic becomes nondeterministic. Consequently, similarity in total scores can conceal different internal weighting: Humans may penalize feasibility due to implementation difficulty while rewarding practical value on other aspects of MRs, whereas ChatGPT and Gemini may reward perceived automability while discounting generalizability and novelty.

Overall, human evaluators produced comparatively balanced results across categories, suggesting that they view MRs as valuable testing hypotheses and still assign value even when some aspects do not strictly satisfy the rubric requirement. ChatGPT exhibited a similar style, which helps explain why its evaluation results remain closer to the human pattern. In contrast, Gemini tended to apply the rubric more strictly, especially when the strict-failure rule is triggered.

B. Divergence between ChatGPT (GPT-5.2) and Gemini (Gemini-3)

Beyond differences in average scores, the LLM evaluators exhibited different *measurement behaviors* that affect how reliably they can be used for MR assessment. In systems with AI components, Gemini-3 applied the rubric most strictly, particularly on *correctness*: Once an MR was judged not clearly checkable, the correctness score dropped sharply, and all the other criteria were consequently driven toward zero, thereby narrowing the spread of total scores. GPT-5.2 more often assigned intermediate scores to partially specified but plausible MRs, preserving score variation and enabling more informative comparisons. This contrast shows that LLM-based MR evaluation can be shaped not only by the selected model, but also by how strongly the model enforces the rubric's baseline conditions: Because the rubric ties several criteria to *correctness*, differences in how strictly evaluators judge correctness can disproportionately shape the overall scores. Strengthening LLM support for more detailed correctness checking—especially for AI-integrated systems—therefore an important future direction for improving the reliability of automated MR evaluation.

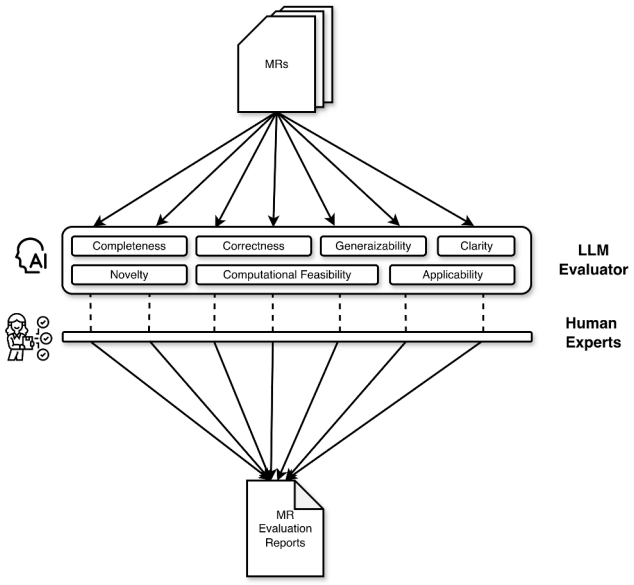


Fig. 3: Framework overview

C. A collaborative framework to combine LLM efficiency and human intelligence

The previous analysis shows that LLMs can efficiently perform MR evaluations, which is particularly beneficial when many MRs must be assessed under tight time constraints. These findings motivate the following workflow that keeps LLM efficiency while ensuring rigorous, rubric-consistent evaluation procedures.

As Fig. 3 shows, LLMs could serve as a first-line evaluator, rapidly assessing and filtering MRs based on broad criteria such as *completeness* and *applicability*. Human experts would then conduct the second-line review, examine correctness for complex SUTs, and apply domain-specific constraints that LLMs may miss. This division of labour would improve efficiency without sacrificing evaluation reliability.

D. Limitations

This study has three main limitations. First, although we considered three SUT categories (comprising nine SUTs) and 27 MRs, this set covers only a small portion of the overall testing space. The selected SUTs were chosen to span increasing levels of complexity, but they cannot capture the full diversity of real-world systems, domains, and practical constraints.

Second, all MRs were generated by ChatGPT. As LLMs evolve, and if MRs are generated by other LLMs, their

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

quality and style may differ, which could shift absolute scores in future replications. Nevertheless, this does not diminish the value of our analysis, which centers on how evaluator behaviors and rubric adherence vary with SUT complexity.

Third, our evaluators may not be representative of all human testers or all LLMs. Although we involved three experienced human evaluators and selected two widely used LLMs (ChatGPT and Gemini), using their free versions to reflect a broader user population, the findings may not generalize to other evaluators or model families.

Overall, these limitations mainly reflect practical resource constraints and do not undermine the contribution of this study. They motivate future work on broader benchmarks, more diverse MR sources, additional evaluator types, and larger-scale analyses with statistical tests, confidence intervals, and inter-rater reliability.

VI. Conclusion

This study proposed a structured rubric for MR evaluation to improve standardization and repeatability. Experiments across three SUT categories show that LLM evaluators align with human experts for low-complexity systems but diverge as complexity increases, particularly on value-oriented criteria and feasibility judgments. We also found clear differences in rule adherence between the LLMs: Gemini applied the rubric most strictly, whereas ChatGPT adopted a more tolerant and human-like scoring style. Overall, the results support a practical workflow where LLMs enable efficient first-pass screening and human experts provide rigorous, context-aware validation for complex or AI-integrated SUTs.

There are three directions for future research. First, the evaluator pool should be expanded to include more LLM families and versions, supporting more robust comparisons across proprietary and open-source models. Second, it should expand the experimental scope with additional SUTs and larger MR sets to better cover diverse domains and complexity levels. Third, it should develop an open benchmark for MR assessment, including curated SUT descriptions, MR sets, and human-validated reference judgments, to support standardized evaluation, reproducibility, and fair comparison of MR evaluators and evaluation rubrics.

Acknowledgments

This work was supported by the 2025 Ningbo Yongjiang Talent–Educational Talent Cultivation Programme, and in part by the Australian Research Council Project under Grant DP260102964.

References

[1] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys (CSUR)*, vol. 51, no. 1, pp. 4:1–4:27, 2018.

[2] H. Liu, F.-C. Kuo, D. Towey, and T. Y. Chen, "How effectively does metamorphic testing alleviate the oracle problem?" *IEEE Transactions on Software Engineering*, vol. 40, no. 1, pp. 4–22, 2014.

[3] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[4] Y. Deng, X. Zheng, T. Zhang, H. Liu, G. Lou, M. Kim, and T. Y. Chen, "A declarative metamorphic testing framework for autonomous driving," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1964–1982, 2022.

[5] Y. Zhang, T. Y. Chen, M. Pike, D. Towey, Z. Ying, and Z. Q. Zhou, "Enhancing autonomous driving simulations: A hybrid metamorphic testing framework with metamorphic relations generated by gpt," *Information and Software Technology*, p. 107828, 2025.

[6] Q. Tao, W. Wu, C. Zhao, and W. Shen, "An automatic testing approach for compiler based on metamorphic testing technique," in *2010 Asia Pacific Software Engineering Conference*. IEEE, 2010, pp. 270–279.

[7] F. Chan, T. Y. Chen, S. C. Cheung, M. F. Lau, and S.-M. Yiu, "Application of metamorphic testing in numerical analysis," in *Proceedings of the IASTED International Conference on Software Engineering (SE'98)*, 1998, pp. 191–197.

[8] Q.-H. Luu, M. F. Lau, S. P. Ng, and T. Y. Chen, "Testing multiple linear regression systems with metamorphic testing," *Journal of Systems and Software*, vol. 182, p. 111062, 2021.

[9] C.-a. Sun, J. Mu, M. Xiao, H. Liu, and P. He, "Semantic structure invariance-based metamorphic testing for machine translation systems," *IEEE Transactions on Reliability*, 2025.

[10] R. Li, H. Liu, P.-L. Poon, D. Towey, C.-A. Sun, Z. Zheng, Z. Q. Zhou, and T. Y. Chen, "Metamorphic relation generation: State of the art and research directions," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–25, 2025.

[11] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, pp. 1–27, 2024.

[12] J. Li, Y. Li, G. Li, Z. Jin, Y. Hao, and X. Hu, "Skcoder: A sketch-based approach for automatic code generation," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 2124–2135. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00179

[13] Y. Zhang, D. Towey, and M. Pike, "Automated metamorphic-relation generation with ChatGPT: An experience report," in *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2023, pp. 1780–1785.

[14] S. Y. Shin, F. Pastore, D. Bianculli, and A. Baicoianu, "Towards generating executable metamorphic relations using large language models," in *International Conference on the Quality of Information and Communications Technology*. Springer, 2024, pp. 126–141.

[15] J. Zhang, C.-a. Sun, H. Liu, and S. Dong, "Can large language models discover metamorphic relations? a large-scale empirical study," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2025, pp. 24–35.

[16] OpenAI, "Introducing GPT-5.2," online, 2025, https://openai.com/index/introducing-gpt-5-2, Accessed: December 21, 2025.

[17] Google DeepMind, "Gemini 3 Flash," online, 2025, https://deepmind.google/models/gemini/flash/, Accessed: December 31, 2025.

[18] Z. Q. Zhou, L. Sun, T. Y. Chen, and D. Towey, "Metamorphic relations for enhancing system understanding and use," *IEEE Transactions on Software Engineering*, vol. 46, no. 10, pp. 1120–1154, 2020.

[19] S. Segura, J. A. Parejo, J. Troya, and A. Ruiz-Cortés, "Metamorphic testing of RESTful Web APIs," *IEEE Transactions on Software Engineering*, vol. 44, no. 11, pp. 1083–1099, 2018.

[20] "Introducing GPTs," online, 2023, https://openai.com/blog/introducing-gpts, Accessed: December 11, 2023.

[21] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 22 199–22 213. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/8bb0d291acd4acf06ef112099c16f326-Paper-Conference.pdf

Appendix
MRs used in this study

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: MRs generated by ChatGPT for nine SUTs across three categories

Type	Category	Description
Basic computational functions	SIN	If the input is x, the output is sin(x). For a new input x + pi, the output should be -sin(x). This tests the periodicity and symmetry of the sine function.
		If the input is x, the output is sin(x). For a new input 2x, the output should follow the identity 2sin(x)cos(x), allowing testing of the sine function over angle doubling.
		If the input is x, the output is sin(x). For a new input x + 2pi, the output should be the same as sin(x), testing the periodic nature of the sine function over a full period.
	SUM	If the input is a list of numbers [a, b, c, ...], the output is their sum S. For a new input where each number in the list is increased by a constant value k ([a+k, b+k, c+k, ...]), the output should be S + n*k, where n is the number of elements in the list. This tests the program's ability to handle uniform increments in the input list.
		If the input list is [a, b, c, ...] with sum S, duplicating any element (e.g., [a, b, c, ..., a]) should result in a new sum of S + a. This tests how the program handles repeated elements.
		Adding zero to the list ([a, b, c, ..., 0]) should not change the sum. If the original sum is S, the new sum should also be S. This tests the program's handling of neutral elements in addition.
	SHORTEST-PATH	For a given graph, if the shortest path is found between two vertices, increasing the weight of one or more edges not in the shortest path should not change the shortest path. This tests the program's handling of irrelevant edge weight changes.
		Adding a new vertex and edges connected to it should not change the shortest path between two existing vertices unless the new edges create a shorter path. This tests the program's adaptability to graph expansion.
		Subdividing an edge (replacing an edge with two edges whose weights sum up to the original edge's weight) should not change the shortest path. This tests the program's handling of graph granularity changes.
Complex systems without AI components	REGRESSION	If the input data rows are scaled by a constant factor, the coefficients should adjust accordingly to produce the same predicted data. Tests uniform data scaling.
		Adding a zero-valued feature should not change the coefficients or predicted data. Tests robustness to irrelevant feature addition.
		Applying inverse transformation to predicted data should align with original scale predictions. Tests consistency across data transformations.
	FFT	Expanding or contracting the time scale should inversely scale frequencies while maintaining amplitudes. Tests time scaling in data.
		Shifting time-series data should not affect frequencies and should impact only the zero frequency amplitude. Tests handling of DC shifts.
		Applying a filter and inverse FFT should result in predictable time-domain changes, reflecting the filter's characteristics.
	WFS	Same weather data from different sources should result in consistent forecasts. Tests data source consistency.
		Scaling input data should result in predictable output changes. Tests response to uniformly scaled data.
		Shifting input data's geographical location should result in an appropriate forecast for the new location. Tests geographical adaptability.
Complex systems with AI components	AV-PERCEPTION	Altering brightness should not significantly change detected objects, testing robustness to lighting variations.
		Scaling images should result in consistent object detection, testing robustness to image scale changes.
		Changing background settings should not affect object detection, testing consistency across environments.
	TRAFFICSYS	Scaling sensor data should lead to proportional traffic decisions, testing response to traffic density variations.
		Omitting sensor data should lead to a conservative traffic response, prioritising safety, testing resilience to incomplete data.
		Detecting emergency vehicles should override regular traffic patterns for prioritisation.
	AUTOPARKING	Changing vehicle size should appropriately adjust parking strategy, e.g., for larger vehicles.
		Shifting surrounding vehicles should result in minor parking manoeuvre adjustments.
		Varying lighting conditions should not significantly impair parking ability, assuming visibility.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on August 29,2026 at 11:29:33 UTC from IEEE Xplore. Restrictions apply.